MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 13 (15-19 Nov 2026) - Period 1	Ch 8 - 8.11 Nested statements

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 13 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

String exit ticket: 80% correct; slicing end index weak. Today: nested selection and iteration (up to 3 levels) before the break.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students write and trace nested statements: selection inside iteration, iteration inside iteration, up to three levels. Syllabus 8.1.5; AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I can trace a nested loop; I can write a loop with an IF inside it; my nested program is correctly indented and ends with ENDIF/NEXT in the right places.

Key knowledge / skill / understanding:

Nesting = a statement inside another; inner loop completes fully for each outer repeat; exam max 3 levels.

Likely misconception / access or challenge need:

Inner loop runs once per program; ENDIF and NEXT can be in any order.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Experiment / lab **Time:** 10 min

What students will do:

Nested loop lab: change the outer and inner ranges of a pattern program (stars, multiplication grid) and record how output changes.

Evidence tool: Observation checklist

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Pattern programs in Python.

Support (low performers / SEN):

Lab sheet with prediction boxes.

Stretch (fast learners / gifted):

Predict total iterations as outer x inner.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 20 min

What students will do:

Worked example then build: exam-results analyser - loop through students, IF mark >= 50 count pass ELSE count fail, plus a nested loop for 3 subjects each.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.11; Python.

Support (low performers / SEN):

Structure diagram showing nesting levels.

Stretch (fast learners / gifted):

Add a third level: CASE for grade letter inside the loops.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Jigsaw **Time:** 7 min

What students will do:

Groups each trace one nested algorithm, then explain the trace to another group.

Evidence tool: Peer-assessment checklist

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Nested algorithm cards; trace tables.

Support (low performers / SEN):

Trace tables with columns set up.

Stretch (fast learners / gifted):

Find the line that makes an algorithm inefficient.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: a clock - minutes loop inside hours loop. Model a nested FOR with an IF inside (syllabus example of row totals), then Python with indentation; colour-code levels on the board.

What is new or innovative in this lesson? *(ETEC looks for this)*

Colour-coded nesting levels and a pattern lab make abstract structure visible.

Progress recorded / shared:

☑ Gradebook ☑ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Mis-indented blocks; inner counter not reset; ENDIF placed after NEXT; more than 3 levels making code unreadable. Likely: 'my grid prints in one line'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit check: how many times does OUTPUT run in a 4 x 3 nested loop? Correct an indentation error. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.11 questions. Before the autumn break: revise loops and selection with the class comparison table.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Pattern lab was popular; students compared Islamic geometric patterns with code patterns. 75% wrote a correct nested program.

What will I change next time?

Colour-coded indentation helped; print it as a handout next time.

Students needing follow-up:

Students below expectations on loops (approx. 3 per class) get a revision pack over the break.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☐3 ☑C ☐H	☑1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 13 (15-19 Nov 2026) - Period 2	Ch 8 - 8.12 Subroutines: procedures and functions

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 13 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Nested exit check: 78% counted iterations correctly. Last lesson before the autumn break: procedures and functions.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students define and call procedures (PROCEDURE ... ENDPROCEDURE, CALL) and functions (FUNCTION ... RETURNS ... ENDFUNCTION) in pseudocode and Python def/return. Syllabus 8.1.6(a)-(b); AO1, AO2.

Success criteria — *what will successful learning look / sound like?*

I can explain procedure vs function; I can write and call a procedure with CALL; I can write a function that RETURNS a value and use it in an expression.

Key knowledge / skill / understanding:

Subroutines reduce repetition and aid testing; a function returns a value; functions are not called with CALL.

Likely misconception / access or challenge need:

Functions print their answer; CALL is used for functions; defining a subroutine runs it.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Discussion / dialogu **Time:** 7 min

What students will do:

Look at a long program with repeated code; discuss what could become a subroutine and why (maintenance, testing, teamwork).

Evidence tool: Oral questioning

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Repetitive program printout.

Support (low performers / SEN):

Highlighted repeated blocks.

Stretch (fast learners / gifted):

Name a real app feature that would be a function.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 21 min

What students will do:

Code along: PROCEDURE DrawLine(Size) with CALL; FUNCTION Area(Length, Width) RETURNS REAL; then refactor the results analyser using one procedure and one function.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.12; syllabus pseudocode p.46-47; Python.

Support (low performers / SEN):

Templates with def lines provided.

Stretch (fast learners / gifted):

Write a function IsValidMark(Mark) RETURNS BOOLEAN.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Think-pair-share **Time:** 8 min

What students will do:

Procedure or function? Pairs classify 6 subroutine descriptions and write the header line in pseudocode.

Evidence tool: Exit ticket

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Classification sheet.

Support (low performers / SEN):

Examples with keyword hints.

Stretch (fast learners / gifted):

Write the CALL or expression that uses each.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 7 min

Hook: a recipe that says 'make the sauce (see page 12)'. Model PROCEDURE Line(Size : INTEGER) and CALL Line(60); then FUNCTION with RETURNS and RETURN, and Python def/return. Stress: no CALL for functions.

What is new or innovative in this lesson? *(ETEC looks for this)*

Students refactor their own earlier program, seeing directly how subroutines make code shorter and easier to maintain.

Progress recorded / shared:

☑ Gradebook ☑ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

print instead of return; calling function without using its value; CALL used with functions; defining after use in Python. Likely: 'my function prints None'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: write a function header that returns the larger of two integers; explain one difference from a procedure. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Over the autumn break (optional): coursebook 8.12 questions 1-3; refactor one earlier program with a subroutine.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Recipe analogy worked across cultures (students shared family recipes). 75% wrote a working function; CALL vs function call clear for most.

What will I change next time?

Give extra practice on return vs print; it was the most common error.

Students needing follow-up:

Students who missed lessons this week: share code notes on Teams before the break.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☑3 ☐C ☐H
Create	**Discussion**	**Pair / group**	**Learner choice**	**Self-assessment**	**Peer assessment**	**SEN / low**	**Gifted**
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback